Tasca S9

Creacion de funciones, estructuras de datos y bucles

Nombre y apellidos: Giorgia Calvagna **Fecha de entrega:** 08/04/2026

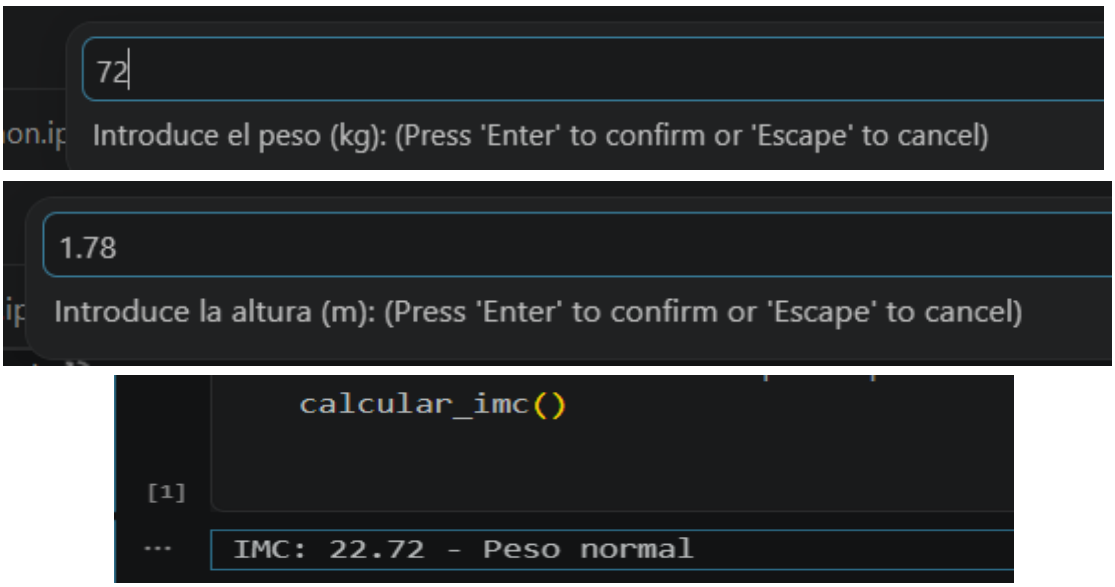
Introducción

Este documento recoge la resolución de los ejercicios de la Tasca S9, cuyo objetivo es aplicar estructuras de control, estructuras de datos (listas, diccionarios) y la definición de funciones en Python para resolver problemas cotidianos. Todos los ejercicios siguen el mismo esquema: se define una función que valida los datos de entrada, realiza el procesamiento y devuelve un resultado guardable, no solo un print().

NIVEL 1

1.1 - Calculadora del Indice de Masa Corporal (IMC)

La función *calcular_imc()* solicita al usuario el peso en kg y la altura en metros mediante *input().* Antes de calcular, valida que los valores sean numéricos con *try/except* y que estén dentro de rangos razonables. El resultado se clasifica en cuatro categorías: Bajo peso, Peso normal, Sobrepeso y Obesidad. La función devuelve el valor del IMC para que pueda guardarse en una variable.



1.2 - Convertidor de temperaturas

La función convertir_temperatura(valor, unidad_origen, unidades destino) recibe una temperatura y su unidad (C, F o K) y devuelve un diccionario con las conversiones a las otras dos unidades. La estrategia clave es convertir primero cualquier temperatura a Celsius como unidad base, y desde ahí calcular el resto con un único diccionario de fórmulas, evitando múltiples bloques if/else para cada combinación posible. La validación comprueba que el valor sea numérico y que la unidad sea reconocida.

```python
# Desde Celsius calculamos todas las conversiones posibles
conversiones = {
    "C": celsius,
    "F": celsius * 9/5 + 32,
    "K": celsius + 273.15
}
```

```python
# Ejecucion
resultado = convertir_temperatura(100, "C")
print(resultado)
```

✓ 0.0s

```
{'F': 212.0, 'K': 373.15}
```

1.3 - Contador de palabras de un texto

La función *contador_palabras(texto)* convierte el texto a minúsculas, elimina los signos de puntuación más comunes con *replace()* y divide el texto en palabras con *split()*. Cuenta las ocurrencias de cada palabra usando un diccionario y el metodo .get(). Como ejercicio extra, calcula la longitud media de las palabras sumando sus caracteres y dividiendo por el total. Devuelve ambos resultados.

```python
# Ejecución
texto_usuario = "Hola, ¿cómo va? Hola de nuevo, te deseo un feliz día!"
conteo, longitud_media = contador_palabras(texto_usuario)

print(f"Frecuencia de palabras: {conteo}")
print(f"Longitud media de las palabras: {longitud_media}")
```

✓ 0.0s

```
Frecuencia de palabras: {'hola': 2, 'cómo': 1, 'va': 1, 'de': 1, 'nuevo': 1, 'te': 1, 'deseo': 1, 'un': 1, 'feliz': 1, 'día': 1}
Longitud media de las palabras: 3.45
```

1.4 - Diccionario inverso (con duplicados)

La función *diccionario_inverso(diccionario)* recorre el diccionario original con items() e intercambia claves y valores. Si un valor aparece más de una vez, convierte la entrada en una lista que agrupa todas las claves asociadas a ese valor e imprime un mensaje de advertencia con los duplicados detectados.

```python
# Ejecucion caso 1: sin duplicados
print(diccionario_inverso({'a': 1, 'b': 2, 'c': 3}))

# Ejecucion caso 2: con duplicados
print(diccionario_inverso({'x': 'apple', 'y': 'banana', 'z': 'banana'}))
```

[4]

```
{1: 'a', 2: 'b', 3: 'c'}
ADVERTENCIA: Se han detectado valores duplicados.
  El valor 'banana' aparece en las claves: ['y', 'z']
{'apple': 'x', 'banana': ['y', 'z']}
```

NIVEL 2

2.1 - Contador y ordenador de palabras desde archivo TXT

La función *contador_organizado(archivo)* abre un fichero .txt, convierte el texto a minúsculas y elimina la puntuación. Ordena todas las palabras alfabéticamente con sorted() antes de construir el diccionario. Esto garantiza que las letras iniciales y las palabras dentro de cada grupo queden en orden A a Z, respetando el orden de inserción en los diccionarios. El resultado es un diccionario anidado con estructura: letra inicial -> {palabra: frecuencia}.

```python
for letra_inicial, grupo_palabras in resultado.items():
    print(f"\n[{letra_inicial.upper()}]")
    for palabra, frecuencia in grupo_palabras.items():
        print(f"  {palabra}: {frecuencia}")
```

```
[A]
  a: 3
  agua: 1
  al: 2
  alba: 4
  alcobas: 1
  alimenta: 1
  alma: 1
  amarga: 1
  azucena: 1

[B]
  baco: 1
  banquete: 1
  bebe: 1
  blanca: 3
```

2.2 - Conversion de tipos de datos

La función *separar_elementos(lista_de_datos)* recorre una lista que puede contener strings, números, listas y tuplas anidadas. Si encuentra una lista o tupla, entra dentro con un segundo bucle for y procesa sus elementos uno a uno. En cada caso intenta la conversión a *float()* con *try/except*: si tiene éxito va a la lista de convertidos; si falla, a la lista de no convertibles. La función devuelve ambas listas.

```python
# Ejecucion
datos = ['1.3', 'one', '1e10', 'seven', '3-1/2', ('2', 1, 1.4, 'not-a-number'), [1, 2, '3', '3.4']]
convertidos, no_convertibles = separar_elementos(datos)

print(f"Elementos convertidos a float: {convertidos}")
print(f"Elementos no convertibles: {no_convertibles}")
```

[6]

```
Elementos convertidos a float: [1.3, 10000000000.0, 2.0, 1.0, 1.4, 1.0, 2.0, 3.0, 3.4]
Elementos no convertibles: ['one', 'seven', '3-1/2', 'not-a-number']
```

NIVEL 3

3.1 - Generador de contrasenas

La función *crear_contrasena(longitud, mayusculas, minusculas, numeros, signos)* construye un conjunto de caracteres disponibles según los parámetros booleanos recibidos, usando el modulo string para obtener los grupos de caracteres sin escribirlos manualmente. La selección aleatoria se realiza con *numpy.random.choice(),* que garantiza aleatoriedad de calidad. Como ejercicio extra, la libreria *pyperclip* copia la contraseña automáticamente al portapapeles. La función devuelve la contraseña como string guardable.

```python
# Probamos la función con 10 caracteres y los parámetros elegidos
mi_nueva_contrasena = crear_contrasena(10, True, True, True, False)

print(f"Tu nueva contraseña es: {mi_nueva_contrasena}")
print("¡Nota: La contraseña ya ha sido copiada a tu portapapeles!")
```

```
Tu nueva contraseña es: hM2Dp5npgn
¡Nota: La contraseña ya ha sido copiada a tu portapapeles!
```

3.2 - Procesamiento de partidos de futbol

La función *analizar_partidos(nombre_archivo)* lee el fichero 'historic partits.txt' linea a linea. Cada línea tiene el formato equipo_local, marcador y equipo_visitante separados por tabuladores, lo que permite nombres de equipo compuestos por varias palabras. El marcador se divide por el guion para extraer los goles de cada equipo. La función mantiene tres diccionarios (goles hechos, goles recibidos y puntos) que se actualizan partido a partido. Al finalizar calcula el equipo mas goleador, el mas goleado y la clasificación ordenada por puntos mediante una función auxiliar o*btener_puntos*.

```python
goles, mas_goleador, mas_goleado, clasificacion = analizar_partidos("historic partits.txt")

print(f"Maximo goleador: {mas_goleador} ({goles[mas_goleador]} goles)")
print(f"Maximo goleado: {mas_goleado}")
print("\n--- CLASIFICACION ---")
for equipo, pts in clasificacion:
    print(f"  {equipo}: {pts} pts")
```

```
Maximo goleador: Figueres (161 goles)
Maximo goleado: Vilafranca

--- CLASIFICACION ---
  Girona FC: 96 pts
  Llagostera: 94 pts
  Sabadell: 85 pts
  Cornellà: 82 pts
  RCD Espanyol: 80 pts
```

Todos los codigos estan disponibles en el repositorio de GitHub.